

# Apache Lab 5 — Command & Expected Output Guide

This PDF preserves the uploaded response content and organizes it for easier command/output understanding. The command blocks and expected outputs are kept as provided.

Yes. Since you already created `example.com` and `webserverlab.com`, first verify that both virtual hosts work over HTTP. Then you can continue with **Lab 5: Securing Apache Web Server**. The lab specifically says these two virtual hosts should already be accessible before starting TLS.

## 1. First check your existing websites

### Step 1 — Check Apache

```
sudo systemctl status apache2
```

If it says **active (running)**, good.

You can also run:

```
sudo apache2ctl configtest
```

You should see:

```
Syntax OK
```

### Step 2 — Check your `/etc/hosts`

Run:

```
cat /etc/hosts
```

You should have something like:

```
127.0.0.1 example.com
127.0.0.1 webserverlab.com
```

If not, edit it:

```
sudo nano /etc/hosts
```

Add:

```
127.0.0.1 example.com
127.0.0.1 webserverlab.com
```

Save:

**Ctrl + O → Enter → Ctrl + X**

### Step 3 — Check DNS/name resolution

```
ping -c 3 example.com
```

and:

```
ping -c 3 webserverlab.com
```

You should see something similar to:

```
PING example.com (127.0.0.1)
```

and:

```
PING webserverlab.com (127.0.0.1)
```

## Step 4 — Check websites using curl

```
curl http://example.com
```

Then:

```
curl http://webserverlab.com
```

If your HTML appears, your virtual hosts are working.

You can also check only the HTTP headers:

```
curl -I http://example.com
```

```
curl -I http://webserverlab.com
```

You want something like:

```
HTTP/1.1 200 OK
```

## Step 5 — Check from browser

Open:

```
http://example.com
```

and:

```
http://webserverlab.com
```

If both show their respective pages, **your previous Web Lab is working correctly.**

## Lab 5 — Securing Apache Web Server

Your uploaded manual calls this:

**CSE-478 Introduction to Computer Security — Lab 5: Securing Apache Web Server.** Its objective is to configure Apache with digital certificates and HTTPS.

There are essentially **3 parts**:

1. Become a Certificate Authority (CA)
2. Create certificates and test HTTPS using OpenSSL
3. Deploy HTTPS into Apache

## Task 1 — Become a Certificate Authority

The lab asks you to create a CA and a self-signed CA certificate.

### Step 1 — Create lab directory

```
mkdir ~/lab5
cd ~/lab5
```

### Step 2 — Copy OpenSSL configuration

```
cp /usr/lib/ssl/openssl.cnf .
```

Check:

```
ls
```

You should see:

```
openssl.cnf
```

The manual specifically instructs you to copy `/usr/lib/ssl/openssl.cnf` into your working directory.

### Step 3 — Check `[ CA_default ]`

Run:

```
grep -A30 "\[ CA_default \]" openssl.cnf
```

Look for directories such as:

```
dir = ./demoCA
database = $dir/index.txt
serial = $dir/serial
```

If it uses `./demoCA`, create:

```
mkdir -p demoCA
touch demoCA/index.txt
echo 1000 > demoCA/serial
mkdir -p demoCA/newcerts
```

Check:

```
ls -R demoCA
```

You should have:

```
demoCA/
■■■ index.txt
■■■ newcerts
■■■ serial
```

## Task 1.1 — Create your Root CA

Run:

```
openssl req -new -x509 -keyout ca.key -out ca.crt -config openssl.cnf
```

The lab manual gives this exact command.

It will ask for a password.

**Remember this password.**

When asked for:

```
Country Name
State or Province Name
Locality Name
Organization Name
Organizational Unit Name
Common Name
Email Address
```

For example:

```
Country Name: BD
State: Dhaka
Locality: Dhaka
Organization: CSE Lab
Organizational Unit: Security
Common Name: CSE Lab Root CA
Email: your-email
```

After completion:

```
ls
```

You should have:

```
ca.key
ca.crt
openssl.cnf
```

## Check certificate

```
openssl x509 -in ca.crt -text -noout
```

Look for:

```
Issuer
Subject
Validity
Public Key
```

For a self-signed CA, **Issuer and Subject refer to the same CA.**

## Task 2 — Create certificate for `example.com`

The manual uses three steps: private key → CSR → CA-signed certificate.

### Step 2.1 — Generate server key

The manual's command is:

```
openssl genrsa -des3 -out server.key 1024
```

It will ask for a password.

Remember it.

Check:

```
ls server.key
```

## Step 2.2 — Generate CSR

Run:

```
openssl req -new -key server.key -out server.csr -config openssl.cnf
```

**Important:** when it asks:

```
Common Name
```

enter:

```
example.com
```

The manual specifically requires `example.com` as the common name.

## Step 2.3 — Sign the certificate with your CA

```
openssl ca -in server.csr -out server.crt -cert ca.crt -keyfile ca.key -config openssl.cnf
```

This is the certificate-generation command from your lab manual.

Now check:

```
ls
```

You should have:

```
ca.key  
ca.crt  
server.key  
server.csr  
server.crt  
openssl.cnf
```

## Task 2.4 — Start HTTPS using OpenSSL

First:

```
cp server.key server.pem
```

Then:

```
cat server.crt >> server.pem
```

Start the OpenSSL web server:

```
openssl s_server -cert server.pem -www
```

The manual says this server listens on **4433** by default.

**Keep this terminal open.**

## Task 2.5 — Test example.com

Open another terminal.

Run:

```
curl -k https://example.com:4433/
```

Or open your browser:

```
https://example.com:4433/
```

Initially, the browser may complain that the certificate is not trusted because your CA is your own CA rather than a browser-trusted commercial CA. This is exactly the behavior described in the lab manual.

## Task 2.6 — Import `ca.crt`

In Firefox:

**Settings → Privacy & Security → Certificates → View Certificates**

Then import:

```
~/lab5/ca.crt
```

Select the option to trust this CA for identifying websites.

The lab specifically asks you to import `ca.crt` and trust it for websites.

Then open:

```
https://example.com:4433
```

## Checkpoint 1

Show your teacher:

```
https://example.com:4433
```

and explain:

*My own CA created a self-signed root certificate. I used that CA to sign the example.com server certificate. After importing the CA certificate into Firefox, the browser trusts the server certificate.*

The manual calls this **Checkpoint 1**.

## Task 2.7 — Do the same for `webserverlab.com`

Stop the OpenSSL server first:

```
ctrl + c
```

Generate another key:

```
openssl genrsa -des3 -out webserverlab.key 1024
```

Generate CSR:

```
openssl req -new -key webserverlab.key -out webserverlab.csr -config openssl.cnf
```

When asked for:

```
Common Name
```

enter:

```
webserverlab.com
```

Sign it:

```
openssl ca -in webserverlab.csr -out webserverlab.crt -cert ca.crt -keyfile ca.key -config openssl.cnf
```

Combine:

```
cp webserverlab.key webserverlab.pem  
cat webserverlab.crt >> webserverlab.pem
```

Start:

```
openssl s_server -cert webserverlab.pem -www
```

Then:

```
https://webserverlab.com:4433
```

This is **Checkpoint 2** in your manual.

## Task 3 — Deploy HTTPS into Apache

Now this is the important part.

First stop OpenSSL:

```
ctrl + c
```

The lab now asks you to put HTTPS configuration into:

```
/etc/apache2/sites-available/example.com.conf
```

and enable Apache SSL.

## Step 3.1 — Enable SSL

```
sudo a2enmod ssl
```

Also enable the SSL site configuration if needed:

```
sudo a2ensite default-ssl
```

## Step 3.2 — Edit example.com configuration

Open:

```
sudo nano /etc/apache2/sites-available/example.com.conf
```

Keep your existing port 80 VirtualHost, and add:

```
<IfModule mod_ssl.c>
<VirtualHost *:443>

    ServerAdmin admin@example.com
    ServerName example.com
    ServerAlias www.example.com

    DocumentRoot /var/www/example.com/html

    ErrorLog ${APACHE_LOG_DIR}/example-error.log
    CustomLog ${APACHE_LOG_DIR}/example-access.log combined

    SSLEngine on
    SSLCertificateFile /home/limon/lab5/server.crt
    SSLCertificateKeyFile /home/limon/lab5/server.key

</VirtualHost>
</IfModule>
```

Since your Ubuntu username is `limon`, I used:

```
/home/limon/lab5/
```

instead of the manual's generic `/path/to/...`.

## Step 3.3 — Create webserverlab HTTPS configuration

Create:

```
sudo nano /etc/apache2/sites-available/webserverlab.com.conf
```

Add your existing HTTP configuration and HTTPS VirtualHost:

```
<IfModule mod_ssl.c>
<VirtualHost *:443>

    ServerAdmin admin@webserverlab.com
    ServerName webserverlab.com
    ServerAlias www.webserverlab.com

    DocumentRoot /var/www/webserverlab.com/html
```

```
ErrorLog ${APACHE_LOG_DIR}/webserverlab-error.log
CustomLog ${APACHE_LOG_DIR}/webserverlab-access.log combined

SSLEngine on
SSLCertificateFile /home/limon/lab5/webserverlab.crt
SSLCertificateKeyFile /home/limon/lab5/webserverlab.key

</VirtualHost>
</IfModule>
```

## Step 3.4 — Enable both sites

```
sudo a2ensite example.com.conf

sudo a2ensite webserverlab.com.conf
```

## Step 3.5 — Test Apache configuration

This is directly required by the lab:

```
sudo apache2ctl configtest
```

You want:

```
Syntax OK
```

The manual specifically identifies ``apache2ctl configtest`` as the configuration test.

## Step 3.6 — Restart Apache

```
sudo systemctl restart apache2
```

Then:

```
sudo systemctl status apache2
```

You want:

```
Active: active (running)
```

## Step 3.7 — Check port 443

Run:

```
sudo ss -tlnp | grep :443
```

You should see Apache listening on:

```
:443
```

## Step 3.8 — Test HTTPS

Now open:

`https://example.com`

and:

`https://webserverlab.com`

Or test from terminal:

`curl -k https://example.com`

`curl -k https://webserverlab.com`

If your HTML appears, Apache HTTPS is working.

The lab requires accessing `https://example.com` for **Checkpoint 3** and `https://webserverlab.com` for **Checkpoint 4**.

## ■ Your complete lab flow

For your situation, remember this order:

```
Existing HTTP sites
↓
example.com ✓
webserverlab.com ✓
↓
Create CA
↓
ca.key
ca.crt
↓
Create example.com certificate
↓
server.key
server.csr
server.crt
↓
Test HTTPS with OpenSSL :4433
↓
Import ca.crt into browser
↓
Create webserverlab.com certificate
↓
Test HTTPS with OpenSSL :4433
↓
Stop OpenSSL
↓
Enable Apache SSL
↓
Configure Apache :443
↓
apache2ctl configtest
↓
Restart Apache
↓
https://example.com
https://webserverlab.com
```

### First, do only these commands now

Since you already created the two domains, **don't recreate them**. Run:

`sudo systemctl status apache2`

`cat /etc/hosts`

```
curl -I http://example.com
```

```
curl -I http://webserverlab.com
```

```
sudo apache2ctl configtest
```

If you paste the output of those **5 commands**, I can tell you exactly whether your existing `example.com` and `webserverlab.com` setup is correct before you start the CA/TLS part.

If you want, I can:

- Share outputs of key checks to verify example.com and webserverlab.com setup
- Help me create and sign certificates for example.com and test with OpenSSL server
- Assist with enabling Apache SSL, configuring HTTPS sites, and testing final HTTPS access